

Học Sâu (Deep Learning)

Chương 4: Bắt đầu với Mạng Nơ-ron (Phân loại và Hồi quy)

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Các thuật ngữ cốt lõi (Glossary)
2. Phân loại Nhị phân (IMDb Dataset)
3. Phân loại Đa lớp (Reuters Dataset)
4. Hồi quy Vô hướng (Boston Housing)
5. Tổng kết

Trong phân loại và hồi quy, các kỹ sư Học máy sử dụng thuật ngữ rất nghiêm ngặt:

- **Sample (Mẫu / Đầu vào):** Một điểm dữ liệu đi vào mô hình. Ví dụ: Một bức ảnh, một đoạn văn bản.
- **Target (Mục tiêu / Sự thật):** Kết quả kỳ vọng. Mô hình lý tưởng sẽ dự đoán ra kết quả này theo một tập dữ liệu chuẩn.
- **Prediction (Dự đoán / Đầu ra):** Giá trị đầu ra thực tế từ mô hình của bạn.
- **Loss value (Giá trị mất mát):** Khoảng cách đo lường sự chênh lệch giữa Prediction (Dự đoán) và Target (Mục tiêu).

Thuật ngữ về Nhãn (Labels & Classes)

- **Classes (Lớp):** Tập hợp các danh mục khả dĩ trong một bài toán phân loại. (Ví dụ: Chó, Mèo, Chim).
- **Label (Nhãn):** Một phiên bản cụ thể của Class được gán cho một Sample. Ví dụ: Ảnh #1234 có nhãn là "Chó".
- **Ground-truth (Sự thật gốc):** Toàn bộ các targets của tập dữ liệu, thường được con người thu thập và gán nhãn thủ công.
- **Batch (Mini-batch - Lô):** Một tập hợp nhỏ các samples (thường 8-128) được mô hình xử lý cùng lúc để tính gradient. Kích thước batch thường là lũy thừa của 2 để tối ưu bộ nhớ GPU.

Phân loại các bài toán Học máy cơ bản

Có 3 bài toán phổ biến nhất trên dữ liệu dạng vector (bảng):

- ➊ **Phân loại Nhị phân (Binary Classification):** Phân loại đầu vào thành 2 nhóm độc quyền (Ví dụ: Chó hoặc Mèo, Positive hoặc Negative).
- ➋ **Phân loại Đa lớp (Multiclass Classification):** Phân loại đầu vào thành nhiều hơn 2 nhóm. Nếu mỗi mẫu chỉ thuộc 1 danh mục duy nhất, ta gọi là bài toán Single-label Multiclass. (Ví dụ: Nhận dạng chữ số 0-9).
- ➌ **Hồi quy Vô hướng (Scalar Regression):** Dự đoán một giá trị mục tiêu liên tục (vô hướng). (Ví dụ: Dự đoán giá nhà, nhiệt độ).

Chuẩn bị dữ liệu (Data Preprocessing)

Mạng nơ-ron không nhận dữ liệu thô (chuỗi văn bản có độ dài khác nhau). Chúng ta phải biến danh sách từ vựng thành tensor:

- **Mã hóa đa điểm (Multi-hot Encoding):** Biến danh sách thành các vector 0 và 1 (chỉ số xuất hiện). Ví dụ: [8, 5] thành vector toàn 0, ngoại trừ chỉ số 5 và 8 là 1.

```
1 import numpy as np
2 def multi_hot_encode(sequences, num_classes):
3     results = np.zeros((len(sequences), num_classes))
4     for i, sequence in enumerate(sequences):
5         results[i, sequence] = 1.0 # Dat 1 tai cac vi tri tu xuat hien
6     return results
```

Bài toán Đánh giá Cảm xúc (IMDb Dataset)

- **Mục tiêu:** Đánh giá xem một bài bình luận phim (movie review) là Tích cực (Positive - 1) hay Tiêu cực (Negative - 0).
- **Dữ liệu:** 50.000 bài đánh giá từ Internet Movie Database, chia đều 25.000 train và 25.000 test. Cân bằng 50% Positive - 50% Negative.
- **Tiền xử lý dữ liệu:** Từ ngữ được mã hóa thành các số nguyên (integer index). Ta sẽ dùng `num_words=10000` để chỉ giữ 10.000 từ xuất hiện phổ biến nhất, loại bỏ các từ quá hiếm.

Tải và Khám phá tập dữ liệu IMDb

```
1 from keras.datasets import imdb
2
3 # Tải 10000 từ phổ biến nhất
4 (train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000)
5
6 # Kiểm tra dòng dữ liệu đầu tiên
7 print(train_data[0])
8 # [1, 14, 22, 16, 43, 530, 973, 1622, ... 178, 32]
9 print(train_labels[0])
10 # 1 (Tích cực)
```

Mỗi bài đánh giá đã được Keras mã hóa sẵn thành chuỗi số nguyên. Ta không phải xử lý văn bản thô (raw text) từ đầu.

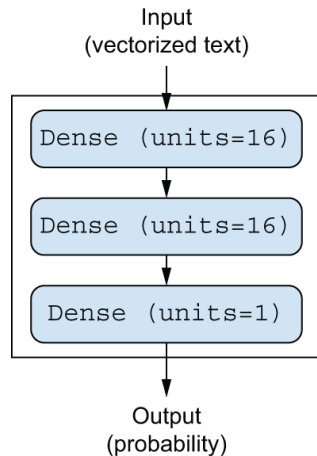
Chuẩn bị Dữ liệu (Vector hóa)

Như đã thảo luận, ta dùng Multi-hot Encoding để biến chuỗi có độ dài thay đổi thành Vector 10000 chiều:

```
1 import numpy as np
2
3 # Vector hóa input (sử dụng hàm multi_hot_encode)
4 x_train = multi_hot_encode(train_data, num_classes=10000)
5 x_test = multi_hot_encode(test_data, num_classes=10000)
6
7 # Label đã là số nguyên 0 hoặc 1, ta chỉ cần chuyển sang float32
8 y_train = train_labels.astype("float32")
9 y_test = test_labels.astype("float32")
```

Kiến trúc Mạng Nơ-ron 3 Lớp

- Mạng có 2 lớp ẩn (Hidden Layers), mỗi lớp có 16 nơ-ron (units).
- Lớp đầu ra (Output Layer) có 1 nơ-ron để đưa ra dự đoán vô hướng (xác suất).
- Ý nghĩa: Lớp thứ nhất trích xuất đặc trưng cơ bản. Lớp thứ hai phân tích quan hệ phức tạp. Lớp cuối tổng hợp kết quả.



Hình 4.1: Cấu trúc mạng 3 lớp cho phân loại IMDb

Định nghĩa Mô hình trong Keras

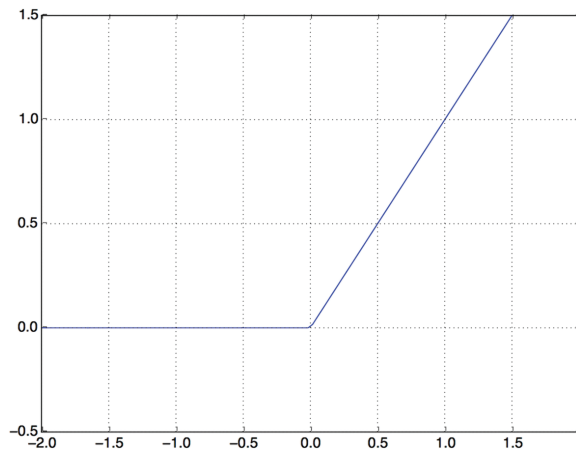
Ta sử dụng `keras.Sequential` để chồng các lớp liên tiếp nhau. Lớp Dense 16 units cần một hàm kích hoạt phi tuyến tính.

```
1 import keras
2 from keras import layers
3
4 model = keras.Sequential([
5     layers.Dense(16, activation="relu"),
6     layers.Dense(16, activation="relu"),
7     layers.Dense(1, activation="sigmoid")
8 ])
```

Hàm kích hoạt (Activation): ReLU

$$f(x) = \max(0, x) \quad (1)$$

- **ReLU (Rectified Linear Unit):** Loại bỏ giá trị âm (trả về 0), giữ nguyên giá trị dương.
- Nếu không có hàm kích hoạt phi tuyến, mạng nhiều lớp cũng chỉ tương đương 1 lớp tuyến tính, không thể giải quyết bài toán phức tạp.

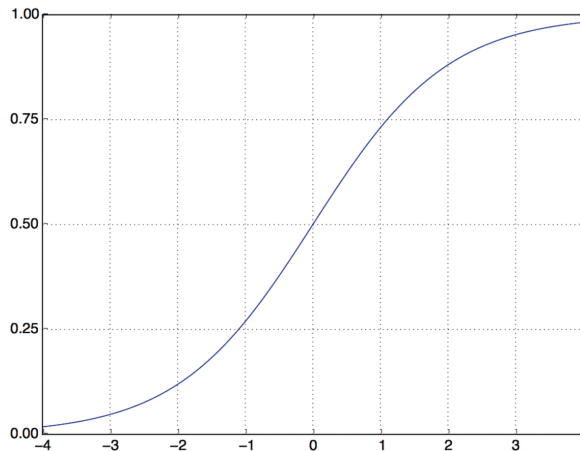


Hình 4.2: Đồ thị của hàm ReLU

Lớp Output: Hàm Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}} \quad (2)$$

- Đầu ra của mạng có thể là số bất kỳ $(-\infty, +\infty)$.
- **Sigmoid:** Ép bất kỳ giá trị nào về miền xác suất $[0, 1]$.
- Ví dụ: Kết quả 0.9 $\rightarrow$ Mô hình có 90% tự tin đây là bình luận Tích cực.



Hình 4.3: Đồ thị của hàm Sigmoid

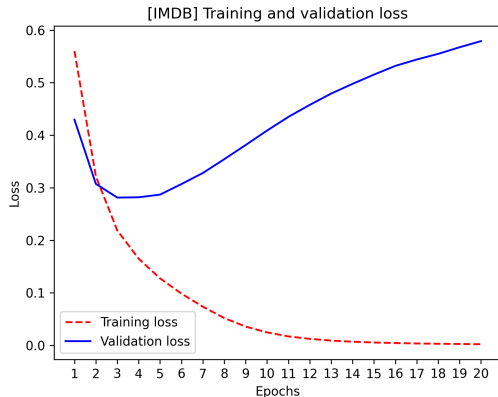
Biên dịch (Compile) và Xác thực (Validation)

Bài toán phân loại nhị phân sử dụng độ đo Loss là `binary_crossentropy`. Thuật toán tối ưu phổ quát là `adam`.

```
1 model.compile(optimizer="adam",
2               loss="binary_crossentropy",
3               metrics=["accuracy"])
4
5 # Huan luyen va theo doi tren tap Validation 20%
6 history = model.fit(x_train,
7                     y_train,
8                     epochs=20,
9                     batch_size=512,
10                    validation_split=0.2)
```

Đánh giá Huấn luyện: Biểu đồ Loss

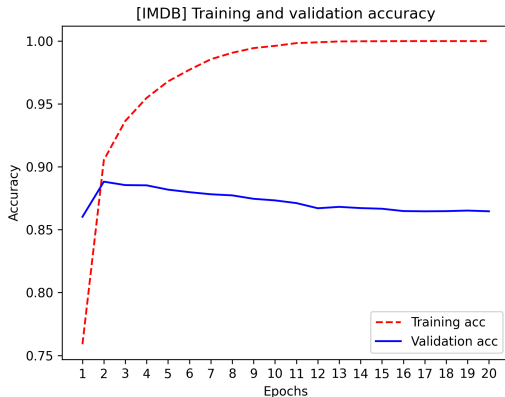
- Đường chấm bi (Training Loss) liên tục giảm → Mô hình đang học rất tốt dữ liệu đào tạo.
- Đường liền (Validation Loss) giảm xuống mức tối thiểu tại epoch 4, sau đó đột ngột tăng lại.
- **Overfitting (Quá khớp):** Mô hình học thuộc lòng dữ liệu train nhưng không có khả năng tổng quát hóa dữ liệu chưa từng thấy.



Hình 4.4: Sự thay đổi Loss qua các Epochs

Đánh giá Huấn luyện: Biểu đồ Accuracy

- Training Accuracy (chấm bi) tiến sát 100% (1.0).
- Validation Accuracy (đường liền) đạt đỉnh khoảng 88% tại epoch thứ 4, sau đó giảm dần.
- **Giải pháp:** Ta nên huấn luyện lại một mô hình mới, cài đặt epochs=4 (Early Stopping) để lấy kết quả tốt nhất.



Hình 4.5: Sự biến thiên của Accuracy

Dự đoán dữ liệu mới (Predictions)

Sau khi có mô hình tốt nhất (train 4 epochs), ta có thể sử dụng hàm `predict()` để đánh giá dữ liệu mới ngoài thực tế.

```
1 predictions = model.predict(x_test)
2 print(predictions)
3 # array([[ 0.98006207],
4 #        [ 0.99758697],
5 #        ...,
6 #        [ 0.02885115],
7 #        [ 0.65371346]], dtype=float32)
```

Nhận xét: Mô hình cực kỳ tự tin với một số mẫu (0.99 - rất Tích cực, hoặc 0.02 - rất Tiêu cực). Nhưng với 0.65, nó còn lưỡng lự.

Bài toán Reuters (Multiclass Classification)

- **Mục tiêu:** Phân loại các bản tin tức (Newswires) của hãng thông tấn Reuters vào 46 chủ đề khác nhau.
- **Đặc trưng bài toán:** Đây là bài toán **Single-label, Multiclass** (Mỗi bài báo chỉ thuộc duy nhất 1 chủ đề, nhưng có tới 46 chủ đề để lựa chọn).
- Dữ liệu có 8982 mẫu huấn luyện và 2246 mẫu kiểm tra.
- **Chuẩn bị:** Dữ liệu đầu vào text cũng được dùng Multi-hot Encoding tương tự như IMDb.

Mã hóa Nhãn (One-Hot Encoding Labels)

Ở IMDb có 2 nhãn (0, 1) nên ta dùng số nguyên. Ở Reuters có 46 chủ đề, ta phải biến đổi mảng nhãn thành dạng Vector qua **One-hot encoding** (Mã hóa phân loại).

```
1 from keras.utils import to_categorical
2
3 # Chuyển [3, 10, 45, ...] thành ma trận 46 cột (0 và 1)
4 y_train = to_categorical(train_labels)
5 y_test = to_categorical(test_labels)
6
7 # y_train[0] có nhãn '3' -> sẽ là vector gồm 45 số 0 và số 1 tại index 3.
```

Kiến trúc Mô hình Đa lớp

- **Thất cổ chai thông tin:** Lớp Dense 16 units là quá nhỏ để truyền tải 46 chủ đề. Thông tin sẽ bị mất mát vĩnh viễn. Ta phải tăng lên 64 units.
- **Lớp đầu ra:** Có 46 units (đại diện 46 chủ đề) kết hợp hàm kích hoạt **Softmax**.

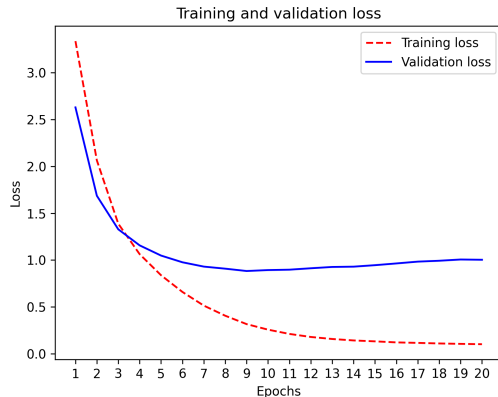
```
1 model = keras.Sequential([  
2     layers.Dense(64, activation="relu"),  
3     layers.Dense(64, activation="relu"),  
4     layers.Dense(46, activation="softmax")  
5 ])
```

Softmax sẽ xuất ra một phân phối xác suất gồm 46 chiều, tổng xác suất luôn bằng 1. Ta cần dùng loss `categorical_crossentropy` để đo khoảng cách giữa 2 phân phối xác suất.

```
1 model.compile(optimizer="adam",  
2               loss="categorical_crossentropy",  
3               metrics=["accuracy"])  
4  
5 history = model.fit(x_train,  
6                   y_train,  
7                   epochs=20,  
8                   batch_size=512,  
9                   validation_split=0.2)
```

Hàm mất mát & Biểu đồ Loss (Reuters)

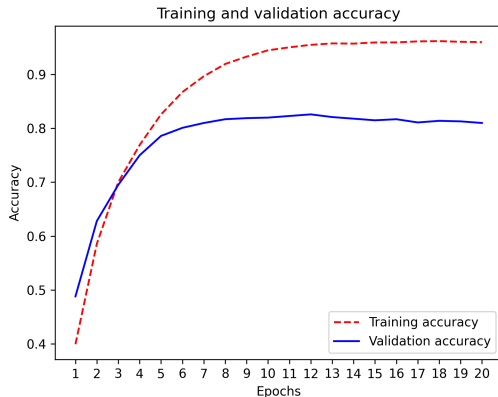
- **Categorical Crossentropy:** Chuyên dụng đo lường khoảng cách giữa hai phân phối xác suất đa chiều.
- Hiện tượng **Overfitting** (Quá khớp) xảy ra ở hầu như mọi bài toán Học sâu. Tại Reuters, Loss trên tập validation bắt đầu tăng mạnh sau epoch thứ 9.



Hình 4.6: Training và Validation Loss (Reuters)

Đánh giá Accuracy

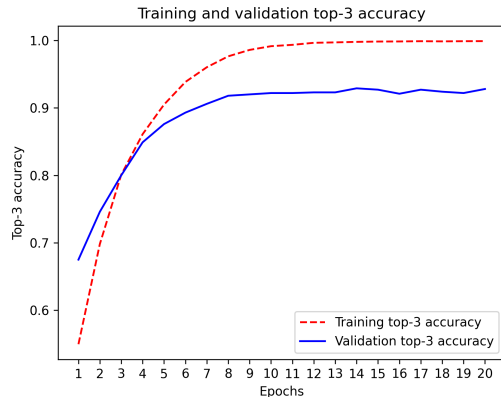
- Accuracy trên tập Validation đạt đỉnh cao nhất khoảng 80% tại epoch thứ 9 trước khi mô hình bị quá khớp.
- So với xác suất chọn ngẫu nhiên (chỉ khoảng 19% nếu đoán bừa theo tần suất tự nhiên của dữ liệu), mức 80% là một kết quả tuyệt vời.



Hình 4.7: Training và Validation Accuracy

Nhưng 80% đã là con số thực? (Top-3 Accuracy)

- Trong tin tức, một bài báo có thể liên quan tới cả 2, 3 chủ đề đan xen. Nếu dự đoán đúng của mô hình là lựa chọn xếp thứ hai hoặc thứ ba thì đó vẫn là thông tin có giá trị.
- Top-3 Accuracy:** Đo lường xem tỷ lệ nhãn thật (Ground-truth) có nằm trong 3 phán đoán có xác suất cao nhất hay không.
- Chỉ số Top-3 này có thể lên tới $>90\%$.



Hình 4.8: Chỉ số Top-3 Accuracy

Bài toán Dự đoán Giá nhà Boston (Scalar Regression)

- **Mục tiêu:** Dự đoán giá nhà trung bình tại vùng ngoại ô Boston (thập niên 1970) dựa trên 13 đặc trưng (tỷ lệ tội phạm, thuế, số phòng...).
- **Khác biệt lớn nhất:** Đầu ra không phải là "xác suất" hay "lớp", mà là **một con số vô hướng liên tục** (Ví dụ: Dự đoán căn nhà giá 25.000 Đô la).
- Tập dữ liệu này cực kỳ nhỏ, chỉ có 506 mẫu. Trong đó 404 mẫu huấn luyện và 102 mẫu kiểm tra.

Tiền xử lý: Chuẩn hóa Dữ liệu (Feature Scaling)

Các đặc trưng dữ liệu (tội phạm, diện tích, tuổi nhà) có thang đo hoàn toàn khác nhau. Đưa trực tiếp vào mạng nơ-ron sẽ gây khó khăn cho hội tụ trọng số. Ta phải chuẩn hóa (**Normalization**): Trừ đi trung bình và chia cho độ lệch chuẩn.

```
1 mean = train_data.mean(axis=0)
2 train_data -= mean
3 std = train_data.std(axis=0)
4 train_data /= std
5
6 # Can phải chuẩn hóa tập test theo trung bình của tập train
7 test_data -= mean
8 test_data /= std
```

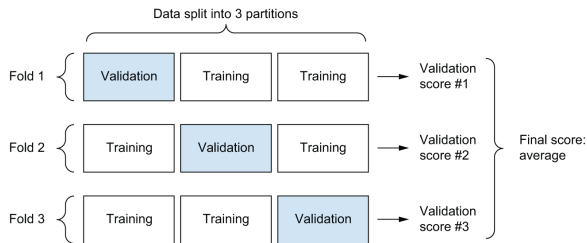
Xây dựng Mạng lưới Hồi quy

Do dữ liệu ít, ta chỉ dùng mạng nơ-ron nhỏ (2 lớp ẩn, 64 units) để tránh Overfitting mạnh.

```
1 def build_model():
2     model = keras.Sequential([
3         layers.Dense(64, activation="relu"),
4         layers.Dense(64, activation="relu"),
5         layers.Dense(1) # Linear Layer - KHÔNG có hàm kích hoạt
6     ])
7     # Loss MSE chuyển để tính lỗi hồi quy
8     # Metric MAE để con người dễ đọc (Mean Absolute Error)
9     model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
10    return model
```

Đặc thù Dữ liệu Nhỏ & K-Fold Cross-Validation

- Tập dữ liệu chỉ có 404 mẫu huấn luyện. Việc tách 100 mẫu ngẫu nhiên ra làm tập Validation sẽ gây ra phương sai sai số rất lớn, phụ thuộc vào "bốc trúng" dữ liệu dễ hay khó.
- **Giải pháp: K-Fold Cross Validation** (Thẩm định chéo K-lần).
- Tách tập train làm K phần bằng nhau ($K = 4$). Mỗi lần dùng 1 phần để validation, 3 phần kia để train. Kết quả là trung bình của K vòng chạy.



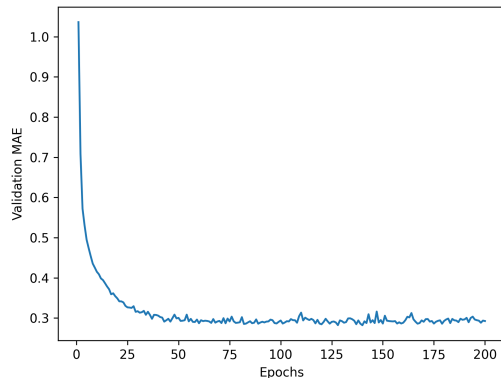
Hình 4.9: Mô phỏng K-Fold Cross-Validation ($K=3$)

Cài đặt K-Fold Validation

```
1 k = 4
2 num_val_samples = len(train_data) // k
3 all_mae_histories = []
4
5 for i in range(k):
6     val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
7     val_targets = train_targets[i * num_val_samples: (i + 1) *
8 num_val_samples]
9     # Ghep du lieu cac fold khac de lam tap huan luyen
10    partial_train_data = np.concatenate([
11        train_data[:i * num_val_samples],
12        train_data[(i + 1) * num_val_samples:]], axis=0)
13    ...
14    # model = build_model()
15    # model.fit(...)
```

Đánh giá Hiệu suất Hồi quy (MAE Plot)

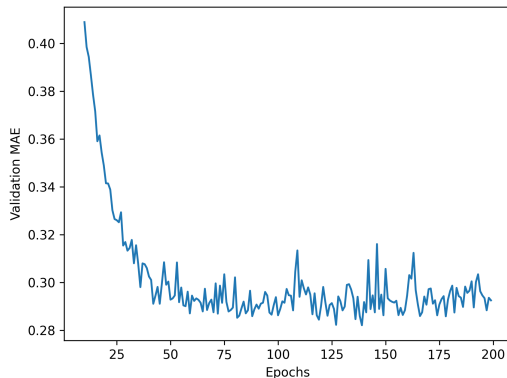
- Khác với Accuracy (càng cao càng tốt), MAE (Mean Absolute Error) biểu thị cho sai số trung bình (càng thấp càng tốt).
 $MAE = 0.5$ tương đương sai lệch 500 Đô la.
- Trực quan hóa giá trị trung bình MAE qua K-folds trên 500 epochs.
- Đường đồ thị có độ rung (nhiều) cao ở những epochs đầu.



Hình 4.10: Validation MAE qua 500 epochs

Phân tích Chuyên sâu (MAE Zoomed Plot)

- Bỏ qua 10 epochs đầu tiên vì sai số quá lớn làm lệch thang đo biểu đồ.
- Vẽ đường làm trơn (Exponential Moving Average) để quan sát rõ hơn xu hướng học.
- Đồ thị chạm đáy tối ưu ở khoảng 120-140 epochs, sau đó bắt đầu tăng vọt → **Overfitting** trên tập dữ liệu hồi quy.



Hình 4.11: Đồ thị làm trơn (Zoomed)

Huấn luyện chung cuộc và Dự đoán

Sau khi xác định được epochs=130 là điểm tốt nhất, ta huấn luyện mô hình trên toàn bộ tập train và chạy test.

```
1 model = build_model()
2 model.fit(train_data, train_targets, epochs=130, batch_size=16, verbose=0)
3
4 test_mse, test_mae = model.evaluate(test_data, test_targets)
5 print(test_mae) # Khoảng 2.5 (Sai lệch khoảng $2,500)
6
7 predictions = model.predict(test_data)
8 print(predictions[0]) # Mang kết quả qua scalar
9 # VD: array([28.34494], dtype=float32) -> Căn nhà có giá 28,344$
```


Bảng tổng kết tra cứu nhanh

Khi bắt đầu dự án Học máy Vector (Bảng), hãy tuân theo quy tắc sau đối với Lớp Output:

Bài toán	Kích hoạt Output	Hàm mất mát (Loss)	Metric
Phân loại nhị phân	sigmoid (1 unit)	binary_crossentropy	accuracy
Phân loại đa lớp	softmax (N units)	categorical_crossentropy	accuracy
Hồi quy vô hướng	Tuyến tính (Linear)	mse (Mean Squared)	mae

- **Tiền xử lý (Preprocessing):** Dữ liệu thô luôn phải được mã hóa dạng One-hot hoặc Chuẩn hóa (Normalization) các đặc trưng không cùng tỷ lệ trước khi đưa vào Mạng Nơ-ron.
- **Nút thắt cổ chai thông tin (Information Bottleneck):** Đối với các mạng đa lớp lớn (như Reuters 46 lớp), nếu chọn lớp ẩn quá nhỏ (vd: 16 units), thông tin sẽ bị đánh rơi vĩnh viễn.
- **Khắc tinh Overfitting:** Mạng nơ-ron LUÔN LUÔN bị quá khớp (overfit) sau một lượng thời gian nhất định. Cần theo dõi biểu đồ Loss trên Validation Set để biết thời điểm nên **Dừng sớm (Early Stopping)**.
- **Dữ liệu quá nhỏ (Small Data):** Rất dễ bị rung lắc mạnh khi tách Validation rồi, hãy nghĩ đến **K-Fold Cross Validation**.